Ahura Agent Architecture

Goal

Transform Ahura from a stateless CLI tool into a stateful, interactive AI Agent.

Core Components

1. Inference Engine (The Brain)

The engine retains the existing OpenRouter integration logic while introducing context awareness. It dynamically constructs request payloads by incorporating historical conversation data to ensure continuity in model responses.

2. Memory Manager (The Storage)

The Memory Manager facilitates persistence by maintaining conversation history within `~/.ahura/sessions/`. By utilizing JSON for data serialization, the system enables users to preserve, switch, and resume specific interaction contexts across different runs.

3. REPL Loop (The Interface)

A centralized Read-Eval-Print Loop (REPL) manages the user interaction lifecycle, featuring: * Input/Output: Streamlined communication between the user and the agent. * Command Parsing: Support for system-level commands including `/reset`, `/save`, `/load`, and `/help`. * Signal Handling: Graceful shutdown procedures triggered via Ctrl+C to ensure session integrity.

Interaction Flow

1. Initialization: User executes `ahura --chat`. The agent attempts to load the most recent session; if none exists, it initializes a new workspace.
2. REPL Loop Execution:
   - Capture: The loop captures raw user input.
   - Contextualization: The system prepends the stored conversation history to the current input to maintain the context window.
   - Inference: The payload is transmitted to the OpenRouter API.
   - Persistence: The agent appends the model's response to the history and flushes the current state to the disk.

Implementation Details

Environment & Compatibility

- Language: Python 3.12+
- Portability: Restricted to the Python Standard Library to ensure cross-platform compatibility and minimal dependencies.

Data Management

- Format: Persistent data is stored as `.json` files.
- Storage Location: `~/.ahura/sessions/`
- File Naming Convention: Sessions are identified using the format `YYYY-MM-DD_ID.json`.